

What Is QS-Net?

Architecture, Notation, and the Three Core Algorithms

Compiled from QuantumSentinel — Novel Methodology and Algorithm (Research Design Document, Version 1.0, June 2026)

1. What QS-Net Is (and Is Not)

QS-Net is short for **QuantumSentinel-Net**. It is the name of the flagship detection method the project is building — a specific software architecture, not a hardware platform. The summary table at the top of the methodology document names it directly:

“Flagship method: QS-Net: Quantum Trust-Region Detector with conformal zero-day rejection”

It runs entirely on a quantum *simulator* (PennyLane's default.mixed device) — there is no physical quantum chip involved in the current project. The document is explicit about this when pre-empting a reviewer objection:

“‘Only simulators, no hardware.’ — We frame simulator-only as a reproducibility strength (deterministic, rerunnable by anyone) and list hardware validation as explicit future work.”

A useful analogy: a **Variational Quantum Circuit (VQC)** is a general technique, the way “convolutional neural network” is a general technique. **QS-Net is a specific system** built on top of a VQC, plus three additional pieces layered on top of it (prototypes, a custom training objective, and a statistical calibration step) — the same way ResNet is a specific architecture built using the general technique of CNNs.

2. The Core Idea, in Plain Language

Section 3.1 of the methodology document explains the geometric intuition behind everything QS-Net does:

“Imagine each known attack type (and benign traffic) forms a cluster of points in a high-dimensional quantum space. We compute the centre of each cluster — a class prototype. Around each prototype we draw a trust ball whose radius we can certify.”

Three consequences follow directly from this picture:

- **An adversarially-disguised known attack** is pushed only a small, bounded distance away from its true cluster centre — it stays inside the trust ball, so it is still classified correctly (robustness).
- **A genuine zero-day attack** lands far from every cluster centre — outside all trust balls — so it is flagged as novel (zero-day detection).
- **The key separation:** because the adversary's perturbation budget is bounded, a disguised known attack cannot travel as far as a true zero-day. That gap is what makes the two distinguishable.

This single geometric object — the trust region — does double duty: it is both the adversarial-robustness margin *and* the zero-day rejection boundary.

3. The Four Building Blocks

Section 3.2 of the document decomposes QS-Net into four components:

Component	What it does (verbatim from the document)
Quantum encoder + VQC	A data re-uploading feature map followed by a variational circuit maps each input into a quantum state $ \Phi(x)\rangle$. Optionally a depolarizing channel models NISQ noise.
Class prototypes	For each known class c , the average (mixed) quantum state of its training samples becomes the prototype density matrix ρ_c — the cluster centre.
Margin-Aware Quantum Training (MAQT)	A training objective that (a) classifies correctly, (b) pulls same-class states toward their prototype, and (c) pushes prototypes of different classes apart — producing tight, well-separated trust regions.
Conformal Zero-Day Rejection (CQ-ZDR)	A split-conformal calibration that sets the rejection threshold so the false-zero-day rate on known attacks is provably bounded by a user-chosen α (e.g., 5%).

4. Notation

The symbols below (Section 3.3) are used throughout the three algorithms that follow.

Symbol	Meaning
x, y	Input feature vector (network flow) and its label (attack class or benign).
$\phi(x) = \psi(x)\rangle$	Quantum encoding (data re-uploading feature map) of the input.
$U(\theta)$	Variational quantum circuit with trainable parameters θ .
Λ_p	Depolarizing noise channel with rate p (NISQ model or deliberate regularizer).
$\rho(x)$	Final (possibly noisy) quantum state: Λ_p applied to $U(\theta)$ acting on $ \psi(x)\rangle\langle\psi(x) $ and its conjugate transpose.
ρ_c	Class prototype: mean quantum state over class- c training samples.
$F(\cdot, \cdot)$	Uhlmann fidelity between two quantum states (1 = identical, 0 = orthogonal).
$D_{\text{tr}}(\cdot, \cdot)$	Trace distance between two quantum states (a metric on states).
L_ϕ	Lipschitz constant of the encoder (how far states move per unit input change).
τ, q	Zero-day rejection threshold (q = conformal threshold for target α).
α	Target maximum false-zero-day rate (e.g., 0.05).
ϵ, R	Adversarial perturbation budget; certified robustness radius.

5. The Three Algorithms

QS-Net is fully specified by three algorithms — training, calibration, and inference. The methodology document notes they are “written so Intern 2, Intern 3, and Intern 4 can implement them directly.” Together they form a pipeline: Algorithm 1 shapes the quantum-state geometry, Algorithm 2 sets a statistically-guaranteed decision threshold using that geometry, and Algorithm 3 uses both at test time to either classify with a certificate or flag a sample as zero-day.

5.1 Algorithm 1 — Margin-Aware Quantum Training (MAQT)

Purpose (verbatim): “Trains the variational circuit so that same-class quantum states cluster tightly and different classes separate, while still classifying correctly. This shapes the Hilbert-space geometry that everything else depends on.”

```
ALGORITHM 1: Margin-Aware Quantum Training (MAQT)
Input : training set  $D = \{(x_i, y_i)\}$ ; encoder  $\phi$ ; VQC depth;
       noise rate  $p$ ; weights  $\lambda_1, \lambda_2$ ; epochs  $E$ ; lr  $\eta$ 
Output: trained parameters  $\theta^*$ ; class prototypes  $\{\rho_c\}$ 
-----
1  initialize  $\theta$  with identity-block init      # barren-plateau safe
2  for epoch = 1 ..  $E$ :
3    for each minibatch  $B$  in  $D$ :
4      for  $x$  in  $B$ :
5         $|\psi(x)\rangle = \phi(x)$                   # encode
6         $|\Phi(x)\rangle = U(\theta) |\psi(x)\rangle$       # variational
7         $\rho(x) = \text{Lambdap}(|\Phi(x)\rangle\langle\Phi(x)|)$  # noise
8        logits = measure( $\rho(x)$ );  $L_{CE} = \text{CrossEntropy}(\text{logits}, y)$ 
9        update running prototype  $\rho_c$  for each class  $c$  in  $B$ 
10        $L_{intra} = \sum_c \text{mean}_{\{x \in c\}} (1 - F(\rho(x), \rho_c))$ 
11        $L_{inter} = - \text{mean}_{\{c \neq c'\}} D_{tr}(\rho_c, \rho_{c'})$ 
12        $L = L_{CE} + \lambda_1 * L_{intra} + \lambda_2 * L_{inter}$ 
13        $\theta \leftarrow \theta - \eta * \text{grad}_{\theta} L$       # parameter-shift
14       log gradient variance; flag barren plateau if  $< 1e-6$ 
15       recompute final prototypes  $\{\rho_c\}$  over all of  $D$ 
16       return  $\theta^*, \{\rho_c\}$ 
```

Reading the loss (line 12): three terms, added together. L_{CE} is ordinary cross-entropy — get the classification right. L_{intra} (line 10) pulls each sample's state toward its own class's prototype (the term is small when the state is close to the prototype, since fidelity F is then close to 1). L_{inter} (line 11) pushes different classes' prototypes apart — maximizing trace distance between them, hence the minus sign, since the overall loss L is being minimized. This is exactly what produces tight, well-separated trust regions, rather than the loosely-scattered clusters an ordinary classifier would learn.

5.2 Algorithm 2 — Conformal Zero-Day Threshold Calibration (CQ-ZDR)

Purpose (verbatim): “Sets the rejection threshold using only known-attack data, with a finite-sample, distribution-free guarantee on the false-zero-day rate. This is the statistical-guarantee centerpiece of the paper.”

ALGORITHM 2: Conformal Zero-Day Calibration (CQ-ZDR)

Input : calibration set D_{cal} (KNOWN classes only);
 prototypes $\{\rho_c\}$; target false-novelty rate α
Output: conformal threshold q

```
-----
1  for each  $x_i$  in  $D_{\text{cal}}$ :
2     $\rho(x_i) = \text{Lambda}_p( U(\theta^*) \text{ phi}(x_i) )$ 
3     $s_i = 1 - \max_c F(\rho(x_i), \rho_c)$           # nonconformity score
4  sort scores  $s_{(1)} \leq s_{(2)} \leq \dots \leq s_{(n)}$ 
5   $k = \text{ceil}( (1 - \alpha) * (n + 1) )$ 
6   $q = s_{(k)}$ 
7  return  $q$ 
   # Guarantee: a fresh known sample is wrongly flagged
   # as zero-day with probability  $\leq \alpha$ .
```

Intuition: the “nonconformity score” s_i is simply 1 minus how well a calibration sample matches its best-fitting prototype. A sample that fits well has a low score; an unusual sample has a high score. Sorting all calibration scores and taking the one at rank $\text{ceil}((1-\alpha)(n+1))$ gives a threshold q with a formal, distribution-free guarantee — this is the classical split-conformal recipe (Vovk, Gammerman & Shafer, 2005), instantiated here with quantum fidelity as the score.

5.3 Algorithm 3 — Unified Inference with Disentangled Rejection

Purpose (verbatim): “At test time, one pass produces either a class label with a certified robustness radius, or a zero-day flag. The same fidelity geometry decides both.”

```

ALGORITHM 3: Unified Inference (detect + disentangle)
Input : test  $x$ ;  $\theta^*$ ; prototypes  $\{\rho_c\}$ ; conformal  $q$ ; noise rate  $p$ ;
        encoder Lipschitz  $L_\phi$ ; constant  $C_f$ 
Output: label in  $\{\text{known classes}\} \cup \{\text{ZERO\_DAY}\}$ ; certified radius  $R$ 
-----
1   $\rho(x) = \text{Lambda}_p( U(\theta^*) \phi(x) )$ 
2  for each  $c$ :  $F_c = F(\rho(x), \rho_c)$ 
3   $c_{\text{star}} = \text{argmax}_c F_c$  ;  $s = 1 - F_{\{c_{\text{star}}\}}$            # novelty score
4  if  $s > q$ :
5      return ZERO_DAY,  $R = 0$                                # outside all trust regions
6  else:
7       $m = F_{\{c_{\text{star}}\}} - \text{second\_max}_c F_c$            # fidelity margin
8       $R = m / ( 2 * (1 - p) * L_\phi * C_f )$            # certified radius
9      return  $c_{\text{star}}$ ,  $R$                                # robust for all  $\|\delta\| \leq R$ 

```

Walking through it: compute the incoming sample's fidelity to every class prototype. Whichever prototype it is closest to (c^*) is the candidate label. If even the best fidelity is too low — i.e. the novelty score s exceeds the conformal threshold q from Algorithm 2 — the sample is rejected as **ZERO_DAY**. Otherwise it is accepted with label c^* , and a **certified radius** R is computed: literally how much input perturbation this decision can tolerate before it might flip, derived from the margin between the best and second-best fidelity match.

6. Why All Three Together Matter

The methodology document summarizes the combination directly, tying the three algorithms to the three supporting theoretical propositions:

“Proposition 2 says adversarial and zero-day samples are separable when the attack budget is below a measurable gap; Proposition 3 says the separating threshold can be set with a guaranteed false-alarm rate; Proposition 1 supplies the quantum-specific contraction term. The combination is a provably-controlled, quantum-native detector — exactly the kind of result that lifts a paper above a benchmark.”

In short, the pipeline is:

Step	What happens	Algorithm
1. Shape the geometry	Train the VQC so same-class states cluster tightly and different classes separate.	Algorithm 1 (MAQT)
2. Set the decision line	Using that geometry, calibrate a rejection threshold with a provable false-alarm-rate guarantee.	Algorithm 2 (CQ-ZDR)
3. Decide at test time	For each new sample, either classify it with a certified robustness radius, or flag it as zero-day.	Algorithm 3 (Inference)

That is the entirety of QS-Net as a system — there is no other hidden machinery. It is a specific quantum-machine-learning architecture (encoder + VQC + prototypes + a custom training loss + conformal

calibration), evaluated entirely on a simulator, aimed at one open problem: telling a disguised known attack apart from a genuinely new one, with a statistical guarantee attached to that decision.

Source: QuantumSentinel — Novel Methodology and Algorithm, Sections 3–5 (Research Design Document, Version 1.0, June 2026).